

Lighting Up the LED

Experiment Objective

In this experiment, you will learn how to control the two LEDs on the MicroROS-V2 control board using the ESP-IDF framework. Through this example, you will master the configuration of GPIO output mode, as well as how to turn the LEDs on, off, and make them blink.

Experiment Principle

1. Keywords

Keyword	Description
GPIO	General-purpose input/output pin, configurable as input or output mode
Push-pull output mode	GPIO output mode that can output a high or low level with strong driving capability
Active high	The LED lights up when the GPIO outputs a high level and goes off when it outputs a low level
Bit mask	Selects the GPIO pins to operate on via binary bits, e.g. <code>(1ULL<<45)</code> selects GPIO45
Blink period	The time for one complete LED on/off cycle, controlled by a timing loop

2. Technical Architecture

The program uses a foreground/background structure: after initializing the GPIO, it enters the main loop and calls the LED blinking function every 10 ms, toggling the GPIO level to make the LEDs blink. The blue LED blinks with a period of 500 ms and the green LED with a period of 1000 ms. Key files are located in `~/esp/base_samples/led/`, including `main/main.c` (main program entry), `components/led/led.h` (pin definitions and function declarations) and `components/led/led.c` (LED driver implementation).

Experiment Steps

1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	 Required
USB Type-C data cable	 Required

2. Prerequisites

1. ESP32-IDF development environment set up
2. MicroROS-V2 control board connected to the PC via USB
3. Confirm the two LEDs (green, blue) on the control board are functioning properly

3. Compile and Flash

Use the command below to check the device serial port number,

```
ls dev/ttyUSB*
```

Compile,

```
cd ~/esp/base_samples/led
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

4. How to Stop

Press `Ctrl+]` in the terminal to exit the serial monitor.

Experiment Result

After compiling and flashing successfully, the serial monitor will output:

```
hello yahboom
I (xxx) LED_MAIN: Nice to meet you!
```

At the same time, the **blue LED** on the control board blinks with a period of about 1 second (500 ms on, 500 ms off), and the **green LED** blinks with a period of about 2 seconds (1000 ms on, 1000 ms off).

Code Explanation

Source code paths:

- Firmware source code: `~/esp/base_samples/led/main/main.c`
- LED driver: `~/esp/base_samples/led/components/led/led.h`, `led.c`

1. Main Program Logic

`main/main.c` is the program entry, and the flow is as follows:

1. Print the startup message `"hello yahboom"` and a welcome log
2. Call `Led_Init()` to initialize the LED GPIO pins
3. Enter the main loop, executed every 10 ms:

- The blue LED toggles its state every 500 ms (`Led_Blue_Flash(50)`), interval of 50×10 ms), i.e. 500 ms on, 500 ms off
- The green LED toggles its state every 1000 ms (`Led_Green_Flash(100)`), interval of 100×10 ms), i.e. 1000 ms on, 1000 ms off

2. Key Components

`components/led/led.h` — pin definitions and function declarations:

Parameter definition file: `~/esp/base_samples/led/components/led/led.h`

```
// GPIO pin definitions
#define LED_GREEN_GPIO      45    // Green LED
#define LED_BLUE_GPIO      48    // Blue LED
#define LED_ACTIVE_LEVEL   1      // Active high to turn on
```

`components/led/led.c` — LED driver implementation, with the following core functions:

Function	Description
<code>Led_Init()</code>	Initialize GPIO45 and GPIO48 as push-pull output mode, with both LEDs initially off
<code>Led_Green_On()</code> / <code>Led_Blue_On()</code>	Turn on the LED of the corresponding color
<code>Led_Green_Off()</code> / <code>Led_Blue_Off()</code>	Turn off the LED of the corresponding color
<code>Led_Green_State(uint8_t state)</code> / <code>Led_Blue_State(uint8_t state)</code>	Set the LED state, 1 to turn on, 0 to turn off
<code>Led_Green_Flash(uint16_t interval)</code> / <code>Led_Blue_Flash(uint16_t interval)</code>	LED blinking function, <code>interval</code> is in units of 10 ms and must be called every 10 ms

Initialization process in detail:

`Led_Init()` uses the `gpio_config_t` structure to configure the GPIO:

Source file: `~/esp/base_samples/led/components/led/led.c`

```
gpio_config_t io_conf = {};
io_conf.pin_bit_mask = (1ULL<<45) | (1ULL<<48); // GPIO45 and GPIO48
io_conf.intr_type = GPIO_INTR_DISABLE;           // Disable interrupt
io_conf.mode = GPIO_MODE_OUTPUT;                 // Output mode
io_conf.pull_down_en = 0;                        // Disable pull-down
io_conf.pull_up_en = 0;                          // Disable pull-up
gpio_config(&io_conf);
```

Common Problems

Problem	Cause	Solution
Compilation fails with "component not found"	ESP-IDF environment not set up correctly	Run <code>source ~/esp/esp-idf/export.sh</code> to initialize the ESP-IDF environment
LED does not light up after flashing	USB cable not connected or wrong serial port number	Check the USB connection and confirm the serial port number; use <code>ls /dev/ttyACM*</code> to list devices
Neither LED blinks	GPIO level configuration does not match the hardware	Check whether the <code>LED_ACTIVE_LEVEL</code> macro in <code>led.h</code> is defined as 1